Food delivery

Grupprojekt

DATASET:FOOD DELIVERY ORDER HISTORY DATA. KAGGLE

UNSUPERVISED LEARNING

Nicklas Thegerström
Emil Sjökvist
Fredrik Käll

Unsupervised learning

- Ladda data, inspektera, kör lite statistik för att börja bekanta oss med datasetet.

```python
import pandas as pd
import numpy as np

import os
os.environ['OMP_NUM_THREADS'] = '1'
```

```python
df = pd.read_csv("data/order_history_kaggle_data.csv")

print("df.info")
print(df.info())
print("df.describe")
print(df.describe())
print("df.head")
print(df.head())

print(f"Före rengöring: {len(df)} rader")
print(f"Saknade värden:\n{df.isnull().sum()}")
```

```
df.info
<class 'pandas.DataFrame'>
RangeIndex: 21321 entries, 0 to 21320
Data columns (total 29 columns):
 #   Column                                          Non-Null Count  Dtype
---  ------                                          --------------  -----
 0   Restaurant ID                                   21321 non-null  int64
 1   Restaurant name                                 21321 non-null  str
 2   Subzone                                         21321 non-null  str
 3   City                                            21321 non-null  str
 4   Order ID                                        21321 non-null  int64
 5   Order Placed At                                 21321 non-null  str
 6   Order Status                                    21321 non-null  str
 7   Delivery                                        21321 non-null  str
 8   Distance                                        21321 non-null  str
 9   Items in order                                  21321 non-null  str
 10  Instructions                                    720 non-null    str
 11  Discount construct                              15823 non-null  str
 12  Bill subtotal                                   21321 non-null  float64
 13  Packaging charges                               21321 non-null  float64
 14  Restaurant discount (Promo)                     21321 non-null  float64
 15  Restaurant discount (Flat offs, Freebies & others)  21321 non-null  float64
 16  Gold discount                                   21321 non-null  float64
 17  Brand pack discount                             21321 non-null  float64
 18  Total                                           21321 non-null  float64
 19  Rating                                          2491 non-null   float64
 20  Review                                          296 non-null    str
 21  Cancellation / Rejection reason                 186 non-null    str
 22  Restaurant compensation (Cancellation)          133 non-null    float64
 23  Restaurant penalty (Rejection)                  3 non-null      float64
 24  KPT duration (minutes)                          21026 non-null  float64
 25  Rider wait time (minutes)                       21153 non-null  float64
 26  Order Ready Marked                              21321 non-null  str
 27  Customer complaint tag                          469 non-null    str
 28  Customer ID                                     21321 non-null  str
dtypes: float64(12), int64(2), str(15)
memory usage: 9.3 MB
None
```

```
df.describe
       Restaurant ID      Order ID  Bill subtotal  Packaging charges  \
count   2.132100e+04  2.132100e+04   21321.000000       21321.000000
mean    2.074413e+07  6.354622e+09     750.870838          32.364592
std     2.447133e+05  1.230633e+08     498.734428          22.235898
min     2.012861e+07  6.086767e+09      50.000000           0.000000
25%     2.063770e+07  6.250752e+09     429.000000          10.450000
50%     2.065987e+07  6.357735e+09     629.000000          28.450000
75%     2.068265e+07  6.450627e+09     899.000000          39.350000
max     2.152306e+07  6.573392e+09   10000.000000         603.000000

       Restaurant discount (Promo)  \
count                 21321.000000
mean                     65.091016
std                      85.481604
min                       0.000000
25%                       0.000000
50%                      30.000000
75%                     100.000000
max                    4020.000000

       Restaurant discount (Flat offs, Freebies & others)  Gold discount  \
count                                       21321.000000       21321.000000
mean                                           31.795058           0.099128
std                                           115.487001           5.364101
min                                             0.000000           0.000000
25%                                             0.000000           0.000000
50%                                             0.000000           0.000000
75%                                             0.000000           0.000000
max                                          7787.000000         288.100000

       Brand pack discount         Total        Rating  \
count         21321.000000  21321.000000  2491.000000
mean              0.039324    682.410133     4.356085
std               17.070788    465.313077     1.101472
min                0.000000     52.500000     1.000000
25%                0.000000    307.450000     4.000000
50%                0.000000    597.450000     5.000000
75%                0.000000    837.900000     5.000000
max              594.300000   12063.000000     5.000000

       Restaurant compensation (Cancellation)  Restaurant penalty (Rejection)  \
count                              133.000000                        3.000000
mean                               356.409549                        0.000000
std                                338.128240                        0.000000
min                                 85.500000                        0.000000
25%                                191.950000                        0.000000
50%                                272.500000                        0.000000
75%                                307.840000                        0.000000
max                                3236.980000                        0.000000

       KPT duration (minutes)  Rider wait time (minutes)
count            21026.000000               21153.000000
mean                17.332960                   4.625670
std                  6.283388                   4.902701
min                  0.000000                   0.100000
25%                 13.350000                   1.000000
50%                 16.330000                   3.100000
75%                 20.010000                   7.400000
max                 90.870000                  73.100000
df.head
   Restaurant ID Restaurant name  Subzone       City   Order ID  \
0       20120607          Sawad  Sector 4  Delhi NCR  6168880918
1       20120607          Sawad  Sector 4  Delhi NCR  6170707559
2       20120607          Sawad  Sector 4  Delhi NCR  6169375019
3       20120607          Sawad  Sector 4  Delhi NCR  6151677434
4       20120607          Sawad  Sector 4  Delhi NCR  6167540897

            Order Placed At Order Status       Delivery Distance  \
0  11:38 PM, September 10 2024   Delivered  Zomato Delivery     3km
1  11:34 PM, September 10 2024   Delivered  Zomato Delivery     2km
2  03:52 PM, September 10 2024   Delivered  Zomato Delivery    <1km
3  03:45 PM, September 10 2024   Delivered  Zomato Delivery     2km
4  03:04 PM, September 10 2024   Delivered  Zomato Delivery     2km

                                                Items in order  ... Rating Review  \
0  1 x Grilled Chicken Jamaican Tender, 1 x Grill...  ...    NaN    NaN
1  1 x Peri Peri Fries, 1 x Fried Chicken Angara ...  ...    NaN    NaN
2                1 x Bone In Peri Peri Grilled Chicken  ...    NaN    NaN
3  1 x Fried Chicken Ghosthunter Tender, 1 x Ange...  ...    4.0    NaN
4  1 x Peri Peri Krispers, 1 x Fried Chicken Ange...  ...    NaN    NaN

   Cancellation / Rejection reason  Restaurant compensation (Cancellation)  \
0                              NaN                                     NaN
1                              NaN                                     NaN
2                              NaN                                     NaN
3                              NaN                                     NaN
4                              NaN                                     NaN

   Restaurant penalty (Rejection)  KPT duration (minutes)  \
0                             NaN                    18.35
1                             NaN                    16.95
2                             NaN                    14.85
3                             NaN                    19.80
4                             NaN                    15.97

   Rider wait time (minutes)  Order Ready Marked  Customer complaint tag  \
0                       11.6           Correctly                     NaN
1                        5.4           Correctly                     NaN
2                       12.2           Correctly                     NaN
3                        3.3           Correctly                     NaN
4                        1.0           Correctly                     NaN

                                        Customer ID
0  5d6c2b96dbd63090bc6f76fdbee5b4c8bf46318a6a0a5178e...
1  8781815deb4a10a574e9fee4fadb6b0874d4a0b36175d5...
2  f9154217cdf362b574d5d1d4e3d81bbacc9c622fc0fc6a...
3  1ed22ddb8a5f7acee12fc1d667555839bc8b2b742af5d...
4  d21a2ac6ce066315c1c3288ab20c4ef2f29286c09f642c5f...

[5 rows x 29 columns]
Före rengöring: 21321 rader
```

```
Saknade värden:
Restaurant ID                                                   0
Restaurant name                                                 0
Subzone                                                         0
City                                                            0
Order ID                                                        0
Order Placed At                                                 0
Order Status                                                    0
Delivery                                                        0
Distance                                                        0
Items in order                                                  0
Instructions                                                20601
Discount construct                                           5498
Bill subtotal                                                   0
Packaging charges                                               0
Restaurant discount (Promo)                                     0
Restaurant discount (Flat offs, Freebies & others)              0
Gold discount                                                   0
Brand pack discount                                             0
Total                                                           0
Rating                                                      18830
Review                                                      21025
Cancellation / Rejection reason                             21135
Restaurant compensation (Cancellation)                      21188
Restaurant penalty (Rejection)                              21318
KPT duration (minutes)                                        295
Rider wait time (minutes)                                     168
Order Ready Marked                                              0
Customer complaint tag                                      20852
Customer ID                                                     0
dtype: int64
```

Unsupervised learning

Datarengöring och EDA

- Fixa datumformat, distansvariabel, saknade värden, några nya features
- Nyckelinsikter: 66% av kunder har bara beställt 1 gang
- Num_items och Total korrelerar starkt, vilket kan vara väntat.

```python
# Kolla exakt hur några datum ser ut
print(df['Order Placed At'].head(10).tolist())

# Datumformatet ser verkligen skumt ut

# Prova parsa manuellt med rätt format
df['Order Placed At'] = pd.to_datetime(
    df['Order Placed At'],
    format='%I:%M %p, %B %d %Y'
)

# Kontrollera att det funkade bra
print(df['Order Placed At'].dtype)
print(df['Order Placed At'].head(3))
```

```
['11:38 PM, September 10 2024', '11:34 PM, September 10 2024', '03:52 PM, September 10 2024',
mber 09 2024', '10:51 PM, September 09 2024', '03:22 PM, September 09 2024']
datetime64[us]
0   2024-09-10 23:38:00
1   2024-09-10 23:34:00
2   2024-09-10 15:52:00
Name: Order Placed At, dtype: datetime64[us]
```

```python
# Meningsfulla features för klustring
df['is_night']   = ((df['Order Placed At'].dt.hour >= 22) |
                    (df['Order Placed At'].dt.hour <= 4)).astype(int)
df['is_weekend'] = (df['Order Placed At'].dt.dayofweek >= 5).astype(int)

df['num_items'] = df['Items in order'].str.count(r'\d+ x')

df['total_discount'] = (
    df['Restaurant discount (Promo)'] +
    df['Restaurant discount (Flat offs, Freebies & others)'] +
    df['Gold discount'] +
    df['Brand pack discount']
)

df['has_discount']  = (df['total_discount'] > 0).astype(int)
df['has_complaint'] = df['Customer complaint tag'].notna().astype(int)
df['was_cancelled'] = df['Order Status'].isin(['Rejected', 'Returned']).astype(int)

df['prep_quality'] = df['Order Ready Marked'].map({
    'Correctly'  : 2,
    'Incorrectly': 1,
    'Missed'     : 0
})
```

```python
# Vi behöver göra något åt distansvariabeln så vi kan få bra km-angivelse

df['Distance_km'] = (df['Distance']
    .str.replace('<', '0.5', regex=False)
    .str.replace('km', '', regex=False)
    .str.strip()
    .astype(float))
```

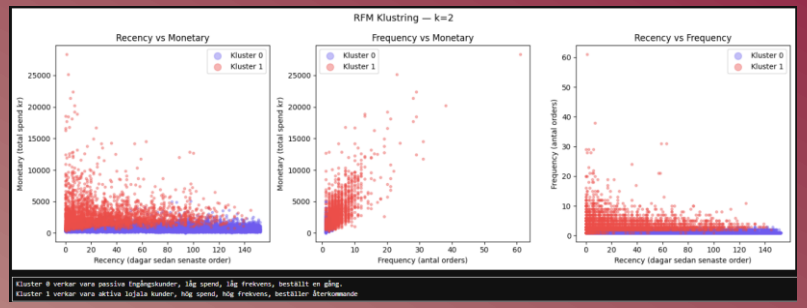
```python
print(df['Distance_km'])
print(df['Distance_km'].describe())
```

```
0        3.00
1        2.00
2        0.51
3        2.00
4        2.00
         ...
21316    5.00
21317    5.00
21318    8.00
21319    9.00
21320    3.00
Name: Distance_km, Length: 21321, dtype: float64
count    21321.000000
mean         4.167815
std          2.994299
min          0.510000
25%          2.000000
50%          3.000000
75%          6.000000
max         21.000000
Name: Distance_km, dtype: float64
```

Unsupervised learning

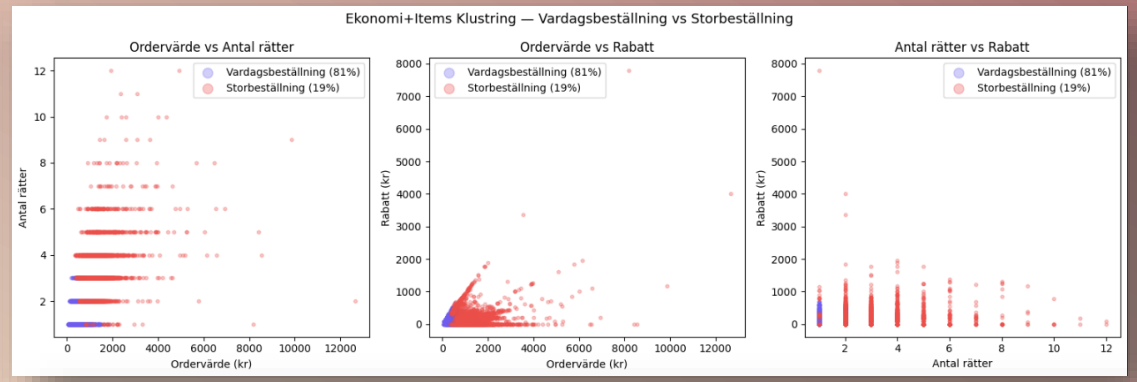
KMeans

- Testade 7 olika feature-kombinationer, se vad som ger högst silhouette-score.
  Bäst var Total + num_items + total_discount
- Resultat: Vardagsbeställning 81%, storbeställning 19%




```python
# Prova olika kombinationer av features för att se om vi kan få upp silhouette-poängen på något sätt

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# lägg till features från df om de inte redan finns i df_ml
df_ml['total_discount'] = df['total_discount']
df_ml['KPT duration (minutes)'] = df['KPT duration (minutes)']
df_ml['Rider wait time (minutes)'] = df['Rider wait time (minutes)']
df_ml['is_night'] = df['is_night']
df_ml['is_weekend'] = df['is_weekend']

# Definiera feature-kombinationer
feature_sets = {
    "Original (3v)": ['Total', 'num_items', 'Distance_km'],

    "Ekonomi+Items": ['Total', 'num_items', 'total_discount'],

    "Logistik": ['KPT duration (minutes)', 'Rider wait time (minutes)', 'Distance_km'],

    "Ekonomi+Logistik": ['Total', 'KPT duration (minutes)', 'Rider wait time (minutes)'],

    "Allt": ['Total', 'num_items', 'Distance_km', 'total_discount',
             'KPT duration (minutes)', 'Rider wait time (minutes)'],

    "Tid+Ekonomi": ['Total', 'is_night', 'is_weekend', 'total_discount'],

    "Items+Logistik": ['num_items', 'KPT duration (minutes)',
                       'Rider wait time (minutes)', 'Distance_km'],
}

print(f"{'Feature set':<20} {'k':<5} {'Silhouette':<12}")
print("-" * 40)

for name, feats in feature_sets.items():
    X_test = df_ml[feats].dropna().copy()
    X_test_scaled = StandardScaler().fit_transform(X_test)

    best_sil, best_k = 0, 0
    for k in range(2, 9):
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        sil = silhouette_score(X_test_scaled, km.fit_predict(X_test_scaled))
        if sil > best_sil:
            best_sil = sil
            best_k = k

    print(f"{name:<20} {best_k:<5} {best_sil:.3f}")
```

```
Feature set          k     Silhouette
----------------------------------------
Original (3v)        2     0.459
Ekonomi+Items        2     0.513
Logistik             2     0.372
Ekonomi+Logistik     2     0.431
Allt                 2     0.332
Tid+Ekonomi          7     0.547
Items+Logistik       2     0.331
```




```python
# Total intäkt per kluster
intakt = df_ml.groupby('cluster_best').agg(
    antal_orders  = ('Total', 'count'),
    total_intakt  = ('Total', 'sum'),
    avg_order     = ('Total', 'mean'),
).round(0)

intakt['andel_orders'] = (intakt['antal_orders'] / intakt['antal_orders'].sum() * 100).round(1)
intakt['andel_intakt'] = (intakt['total_intakt'] / intakt['total_intakt'].sum() * 100).round(1)

print(intakt)
```

```
             antal_orders  total_intakt  avg_order  andel_orders  \
cluster_best
0                   17273     9304786.0      539.0          81.0
1                    4048     5249272.0     1297.0          19.0

             andel_intakt
cluster_best
0                    63.9
1                    36.1
```

Vad vi hittade: KMeans med k=2 på ordervärde, antal rätter och rabatt identifierade två distinkta ordertyper med silhouette score 0.513, verkar bra!

De två segmenten: Vardagsbeställningen (81% av orders)

Enkel, snabb, billig — 539 kr, 1-2 rätter Troligen en person som beställer lunch eller middag Liten men proportionell rabatt (15%)

Storbeställningen (19% av orders)

Komplex, dyr — 1 297 kr, 3+ rätter Troligen grupp, familj eller arbetsplats Samma rabattandel (14%) men dubbelt så mycket i kronor

Det viktigaste fyndet, rabattsystemet är blindt: Plattformen ger rabatt proportionellt oavsett ordertyp. Det är suboptimalt eftersom de två segmenten har helt olika priskänslighet och lojalitetspotential. En storbeställare som tar med hela kontoret är värd mer att behålla än en ensambeställare.

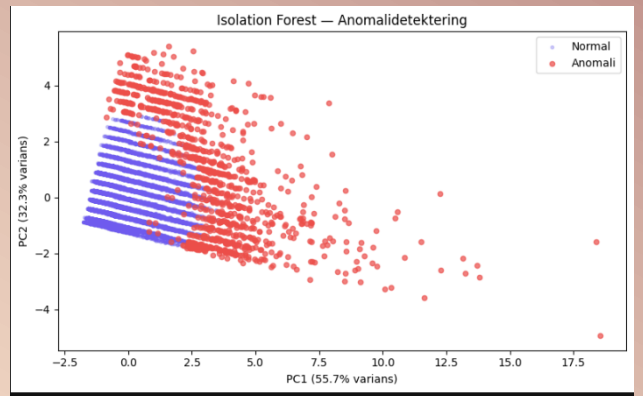
Trots dåligt kundunderlag med mycket engångsbeställare kan vi väl testa att göra RFM och försöka segmentera lite kunder!

Unsupervised learning

Isolation forest

- Identifierar anomalier beserat på hur lätt de isoleras I beslutsträd.
- Features: Total, num_items, Distance_km
- Resultat: 1049 anomalier (5%), Auras Pizzas dominant
- Insikt: Anomalier är en grupp med väsentligt högre ordervärde och längre leveransavstånd

```python
# PCA-plot med anomalier markerade
plt.figure(figsize=(8,5))
normal_mask   = df_ml['anomaly'] == 1
anomaly_mask  = df_ml['anomaly'] == -1

plt.scatter(X_pca[normal_mask, 0],  X_pca[normal_mask, 1],
            c='#6366f1', alpha=0.3, s=8,  label='Normal')
plt.scatter(X_pca[anomaly_mask, 0], X_pca[anomaly_mask, 1],
            c='#ef4444', alpha=0.7, s=20, label='Anomali')

plt.title('Isolation Forest — Anomalidetektering')
plt.xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} varians)')
plt.ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} varians)')
plt.legend()
plt.tight_layout()
plt.show()

# Vilka restauranger har flest anomalier?
df_with_anomaly = df.loc[df_ml.index]
df_with_anomaly['anomaly'] = df_ml['anomaly']

print("\nAnomalier per restaurang:")
print(df_with_anomaly.groupby('Restaurant name')['anomaly']
      .apply(lambda x: (x == -1).sum())
      .sort_values(ascending=False))

print("\nAnomalirate per restaurang:")
print(df_with_anomaly.groupby('Restaurant name')['anomaly']
      .apply(lambda x: (x == -1).mean())
      .round(3)
      .sort_values(ascending=False))
```



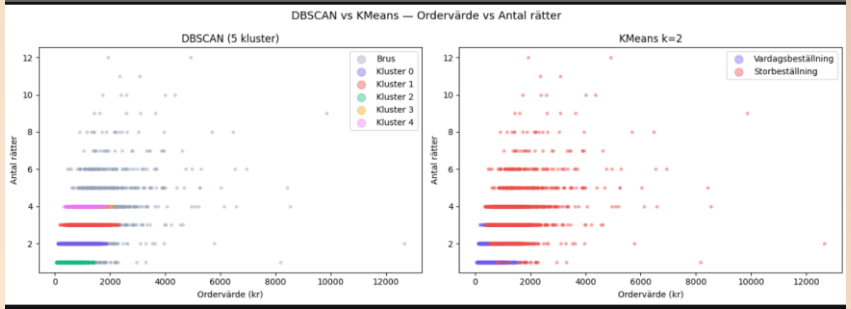

```
Anomalier per restaurang:
Restaurant name
Aura Pizzas             862
Swaad                   166
Tandoori Junction        17
The Chicken Junction      9
Masala Junction           7
Dilli Burger Adda         5
Name: anomaly, dtype: int64

Anomalirate per restaurang:
Restaurant name
The Chicken Junction    0.281
Masala Junction         0.250
Tandoori Junction       0.110
Aura Pizzas             0.059
Swaad                   0.026
Dilli Burger Adda       0.022
```

Unsupervised learning

DBSCAN vs KMeans

- DBSCAN kräver inte fördefinierat k, hittar oregelbundna kluster och identifierar brus automatiskt
- DBSCAN definierar 4 ratter och sätter resten som brus, KMeans mer användbart I vårt fall

```python
# Visualisera DBSCAN vs KMeans
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('DBSCAN vs KMeans — Ordervärde vs Antal rätter', fontsize=13)

# DBSCAN
dbscan_colors = {-1: '#94a3b8', 0: '#6366f1', 1: '#ef4444',
                 2: '#10b981', 3: '#f59e0b', 4: '#e879f9'}
for c in sorted(df_ml['dbscan_cluster'].unique()):
    mask = df_ml['dbscan_cluster'] == c
    label = 'Brus' if c == -1 else f'Kluster {c}'
    axes[0].scatter(df_ml[mask]['Total'], df_ml[mask]['num_items'],
                    c=dbscan_colors.get(c, '#94a3b8'),
                    alpha=0.4, s=10, label=label)
axes[0].set_xlabel('Ordervärde (kr)')
axes[0].set_ylabel('Antal rätter')
axes[0].set_title('DBSCAN (5 kluster)')
axes[0].legend(markerscale=3)

# KMeans
kmeans_colors = {0: '#6366f1', 1: '#ef4444'}
for c in [0, 1]:
    mask = df_ml['cluster_best'] == c
    label = 'Vardagsbeställning' if c == 0 else 'Storbeställning'
    axes[1].scatter(df_ml[mask]['Total'], df_ml[mask]['num_items'],
                    c=kmeans_colors[c], alpha=0.4, s=10, label=label)
axes[1].set_xlabel('Ordervärde (kr)')
axes[1].set_ylabel('Antal rätter')
axes[1].set_title('KMeans k=2')
axes[1].legend(markerscale=3)

plt.tight_layout()
plt.show()
```



DBSCAN klustrar 1, 2 3 och fyra rätter och behandlar resten som brus! KMEANS segmenterar på ekonomiskt värde